

Lab 1: The Rental Refund Crisis

Scenario: A customer returns a damaged DVD.

We need to:

1. Update the rental status to 'returned'.
2. Charge a damage fee to the payment table.
3. If any step fails, no data should change.

```
234  -- ex 1
235  SELECT rental_id, customer_id FROM rental LIMIT 1;
236
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	rental_id [PK] integer	customer_id smallint
1	2	459

Exercise 1: Testing the "All or Nothing"

```
16  begin;
17
18  update rental
19  set return_date = now()
20  where rental_id = 2;
21
22  insert into payment (customer_id, staff_id, rental_id, amount, payment_date)
23  values (459, 1, 2, 9.99, now());
24
25  commit;
26
27  -- verify it worked:
28  select rental_id, return_date from rental where rental_id = 2;
29  select * from payment where rental_id = 2 order by payment_date desc limit 1;
30
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)	payment_date timestamp without time zone
1	32099	459	1	2	9.99	2026-09-18 14:56:24.778795

ANS: return_date không đổi, vì Step 2 lỗi khiến cả transaction bị đánh dấu **aborted** - COMMIT lúc đó tự động thành ROLLBACK, hủy luôn cả Step 1 dù nó chạy ổn. Transaction chỉ tất cả hoặc không gì cả

Exercise 2: The "Flexible" Staff Task

```
59 begin;
60
61 -- 1. register a new staff member.
62 --   store_id is not null + has a foreign key, so we must start with a
63 --   valid store (store 1) as a placeholder before we test the bad one.
64 insert into staff (first_name, last_name, address_id, email, store_id, active, username
65 values ('an', 'nguyen', 1, 'an.nguyen@dvdrental.local', 1, true, 'an_nguyen');
66
67 -- 2. set a savepoint right after the staff member exists
68 savepoint staff_created;
69
70 -- 3. try to assign them to a non-existent store_id -> this fails
71 --   (violates the fk: staff.store_id -> store.store_id)
72 update staff
73 set store_id = 9999
74 where username = 'an_nguyen';
75
76 -- 4. it failed -> roll back only to the savepoint (staff row survives),
77 --   then assign them to store 1 instead
78 rollback to savepoint staff_created;
79
80 update staff
81 set store_id = 1
82 where username = 'an_nguyen';
83
84 -- 5. commit everything
85 commit;
86
87 -- verify:
88 select staff_id, first_name, last_name, store_id from staff where username = 'an_nguyen';
89
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

	staff_id [PK] integer	first_name character varying (45)	last_name character varying (45)	store_id smallint
1	3	an	nguyen	1

ANS: Exercise 2 (Savepoint): Tạo staff -> SAVEPOINT -> update sang store lỗi (FK fail) -> ROLLBACK TO SAVEPOINT chỉ lùi về mốc đó, giữ lại staff vừa tạo (không hủy cả transaction) -> gán lại store đúng -> COMMIT

Durability: Where is the Log?

```
92 -- durability: the "crash" test
93
94 -- note the original value first, so you can restore it later:
95 select customer_id, first_name from customer where customer_id = 1;
96
97 begin;
98 update customer set first_name = 'durabilitytest' where customer_id = 1;
99 commit;
100
101 -- now: task manager -> end task on all postgres.exe processes (simulate
102 -- a crash) -> restart the postgresql service -> then run:
103 select first_name from customer where customer_id = 1;
```

Data Output Messages Notifications



Showing rows: 1 to 1 Page No: 1 of 1

	first_name character varying (45)
1	durabilitytest